Operadores fisicos: Sorting

Clase 17

IIC2413 - Sección 3

Prof. Cristian Riveros

Outline

Reglas de reescritura (anterior)

Operadores físicos

Merge sort

External merge sort

Optimizaciones

Outline

Reglas de reescritura (anterior)

Operadores físicos

Merge sort

External merge sort

Optimizaciones

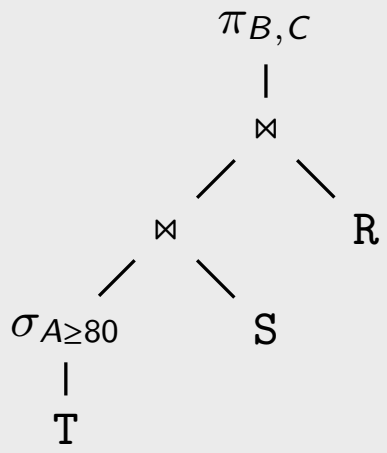
Plan lógico (recordatorio)

Definición

Un **plan lógico** es un árbol ordenado y etiquetado tal que:

- los nodos **hojas** son relaciones y
- los nodos **internos** son operadores del álgebra relacional (extendido)

Ejemplo de plan lógico

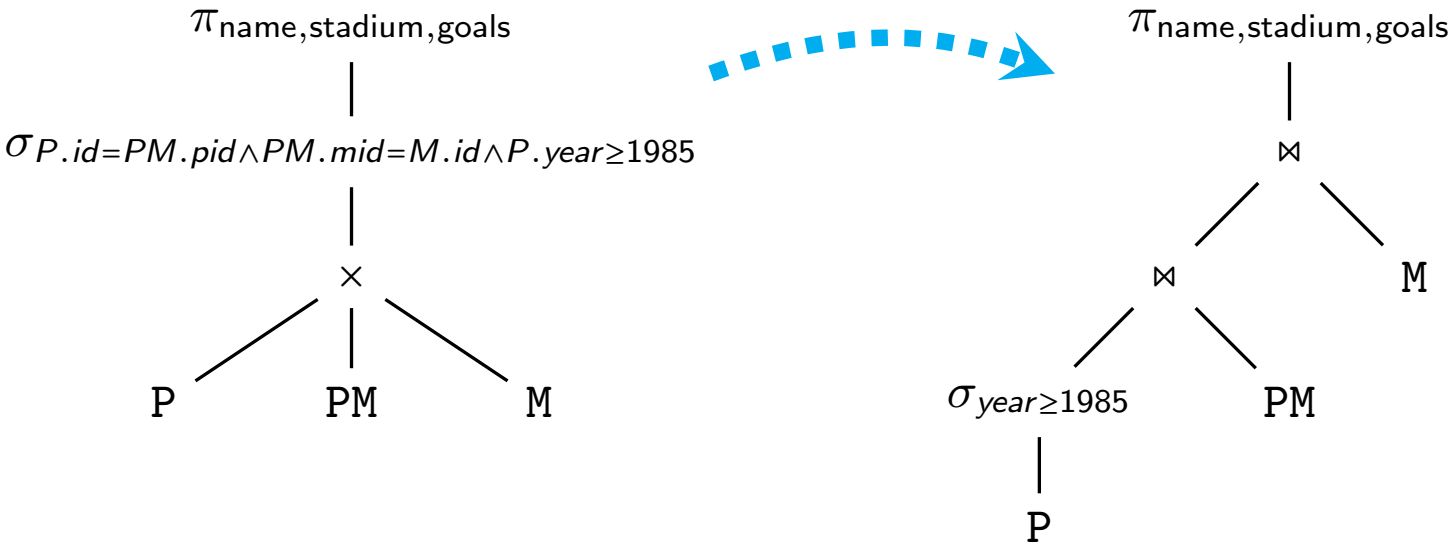


Plan lógico define la **semántica de la consulta SQL**

(y no como computar la consulta)

Reescritura de consultas (recordatorio)

1. Convertir el árbol de parsing en un plan lógico.
2. Reescribe consulta aplicando reglas de álgebra relacional.
3. Crea un set de planes lógicos **prometedores** para ser optimizados.



Reglas de reescritura de consultas (recordatorio)

1. Conmutatividad ✓
2. Asociatividad ✓
3. Distributividad ✓
4. Distribución de selección (push-selection) ✓
5. Distribución de proyección (push-projection)
6. Simplificación de productos
7. Eliminación de duplicados

Una de las **mayores ventajas** del álgebra relacional

Selección: push-selection (recordatorio)

Teorema

Si R y S tiene todos los atributos mencionados en p, simultáneamente:

$$\begin{aligned} \sigma_\theta(R \bowtie S) &= \sigma_\theta(R) \bowtie \sigma_\theta(S) \\ \sigma_\theta(R \cap S) &= \sigma_\theta(R) \cap \sigma_\theta(S) \end{aligned}$$

Las siguientes consultas son siempre **equivalentes**:

$$\begin{aligned} \sigma_\theta(R \cup S) &= \sigma_\theta(R) \cup \sigma_\theta(S) \\ \sigma_\theta(R \setminus S) &= \sigma_\theta(R) \setminus \sigma_\theta(S) = \sigma_\theta(R) \setminus S \end{aligned}$$

Una de las reglas **mas importante** de los motores

Proyección: push-projection

Teorema

Las siguientes consultas son **equivalentes**:

$$\pi_L(R \bowtie S) = \pi_L(\pi_M(R) \bowtie \pi_N(S))$$

$$\pi_L(R \bowtie_\theta S) = \pi_L(\pi_M(R) \bowtie_\theta \pi_N(S))$$

$$\pi_L(R \times S) = \pi_L(\pi_M(R) \times \pi_N(S))$$

$$\pi_L(R \cup S) = \pi_L(R) \cup \pi_L(S)$$

$$\pi_L(\sigma_\theta(R)) = \pi_L(\sigma_\theta(\pi_M(R)))$$

si M y N no **interfieren** con el join ni la proyección de L.

Ejemplo: $R(a, b, c)$ y $S(b, c, d)$

$$\pi_{a,b}(R \bowtie S)$$

Joins y productos

Teorema

Las siguientes consultas son **equivalentes**:

$$R \bowtie_\theta S = \sigma_\theta(R \times S)$$

$$R \bowtie S = \pi_L(\sigma_\alpha(R \times S))$$

donde:

$$\alpha := \bigwedge_{A \in \mathrm{att}(R) \cap \mathrm{att}(S)} R.A = S.A$$

$$L := \mathrm{att}(R) \cup \mathrm{att}(S).$$

¿cuál es la **utilidad** de esta regla?

Eliminación de duplicados

Teorema

Las siguientes consultas son **equivalentes**:

$$\begin{aligned}
\delta(R \times S) &= \delta(R) \times \delta(S) \\
\delta(R \bowtie S) &= \delta(R) \bowtie \delta(S) \\
\delta(R \bowtie_p S) &= \delta(R) \bowtie_p \delta(S) \\
\delta(\sigma_p(R)) &= \sigma_p(\delta(R))
\end{aligned}$$

Siempre considerar que $\delta(R) = R$ si R **NO tiene duplicados** como:

- R tiene una llave primaria.
- R es el resultado de un group-by.
- R es el resultado de operadores sin duplicados.

GroupBy

- Reglas dependen mucho de las funciones de agregación.
- Algunas reglas:

$$\delta(\gamma_L(R)) = \gamma_L(R)$$

$$\gamma_L(\delta(R)) = \gamma_L(R) \quad (*)$$

$(*)$ si L considera funciones de agregación como `MAX` o `MIN`.

¿qué reglas usar?

No existe una estrategia única

Heurísticas

- hacer **push** de las **selecciones** lo mas abajo posible
- hacer **push** de las **proyecciones** lo mas abajo posible
- proyectar y eliminar atributos que no se usarán
- convertir los productos en **equijoins**
- elegir el **mejor orden*** de joins, uniones o intersecciones
- remover eliminación de duplicados innecesarios

Outline

Reglas de reescritura (anterior)

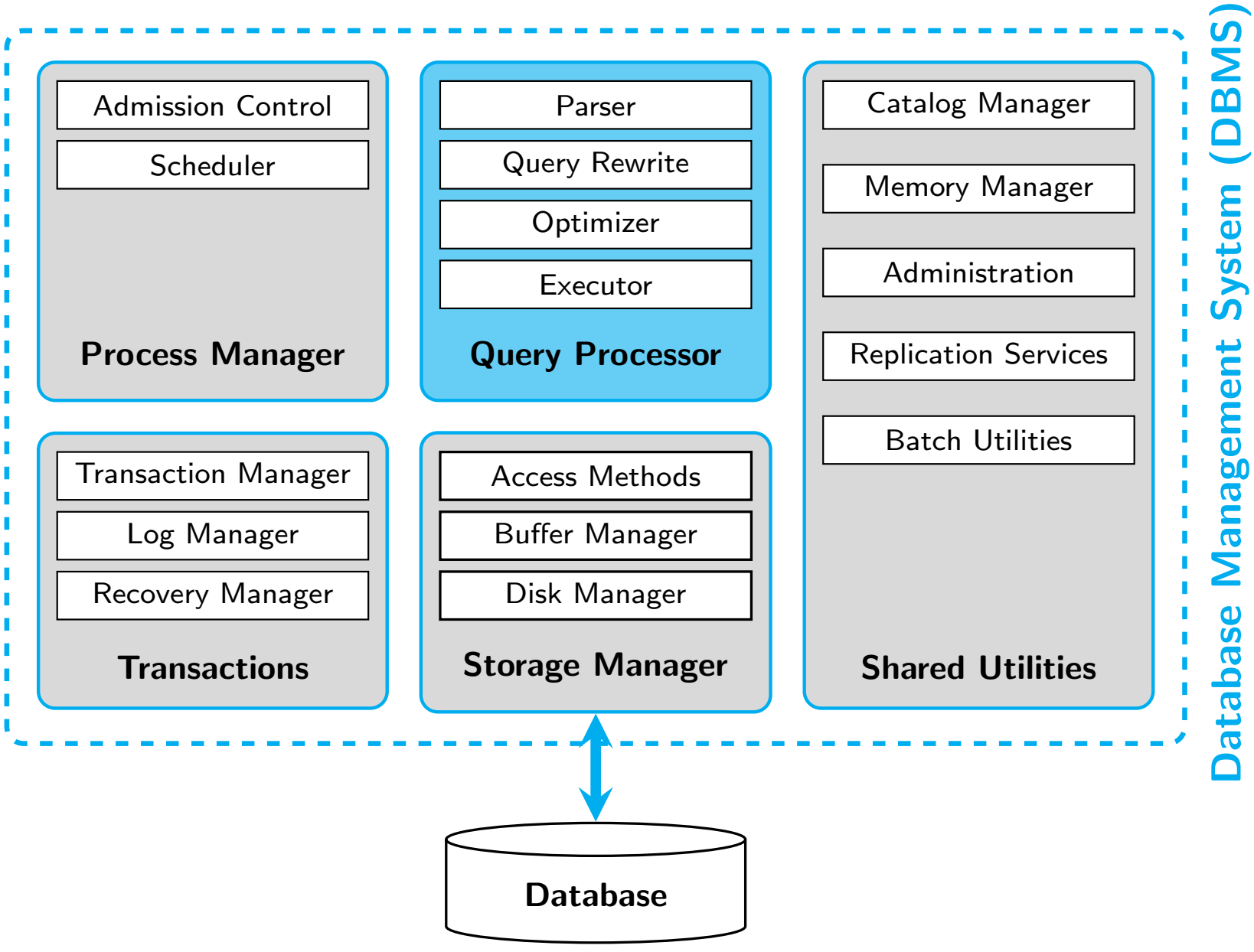
Operadores físicos

Merge sort

External merge sort

Optimizaciones

Implementación de operadores relacionales



Implementación de operadores relacionales

¿qué es un operador físico?

- Cada operador físico implementa un operador relacional (lógico).
- Orientado a desempeñarse bien en una tarea específica.

Cada **variante** aprovecha propiedades físicas de los datos:

- Presencia o ausencia de índices.
- Orden del input.
- Tamaño del input.
- Cantidad de elementos distintos.
- Espacio disponible en memoria.

Operadores físicos relacionales

Selección (σ)

Proyección (π)

Join ($\bowtie$)

Intersección ($\cap$)

Unión ($\cup$)

Elim. duplicados (δ)

Sorting (τ)

GroupBy (γ)

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

¿por qué necesitamos sorting?

- Operador de `SQL`.

Ejemplo

```
SELECT name, position
FROM Players
ORDER BY position
```

- Para implementar otros operadores:
  - Eliminación de duplicados (`DISTINCT`)
  - Joins
  - Intersección
  - Group by

- Uso para construir **índices** y otras estructuras.

Sorting es un operador básico dentro del funcionamiento de una DBMS

¿por qué queremos estudiar sorting?

Sorting externo

¿cómo podemos ordenar un conjunto de tuplas
cuyo tamaño excede **por mucho** *el espacio disponible en RAM?*

External merge sort

¿cómo funciona el algoritmo **merge sort**?

Outline

Reglas de reescritura (anterior)

Operadores físicos

Merge sort

External merge sort

Optimizaciones

Merge sort



Merge sort: listas ordenadas

¿cómo ordenar dos listas ordenadas en una sola lista?

Lista 1	Lista 2	
$4, 7, 17, 23$	$1, 9, 10, 15$	
$4, 7, 17, 23$	$9, 10, 15$	1
$7, 17, 23$	$9, 10, 15$	$1, 4$
$17, 23$	$9, 10, 15$	$1, 4, 7$
$17, 23$	$10, 15$	$1, 4, 7, 9$
$17, 23$	15	$1, 4, 7, 9, 10$
$17, 23$		$1, 4, 7, 9, 10, 15$
		$1, 4, 7, 9, 10, 15, 17, 23$

Merge sort: listas ordenadas

Algoritmo de Merge

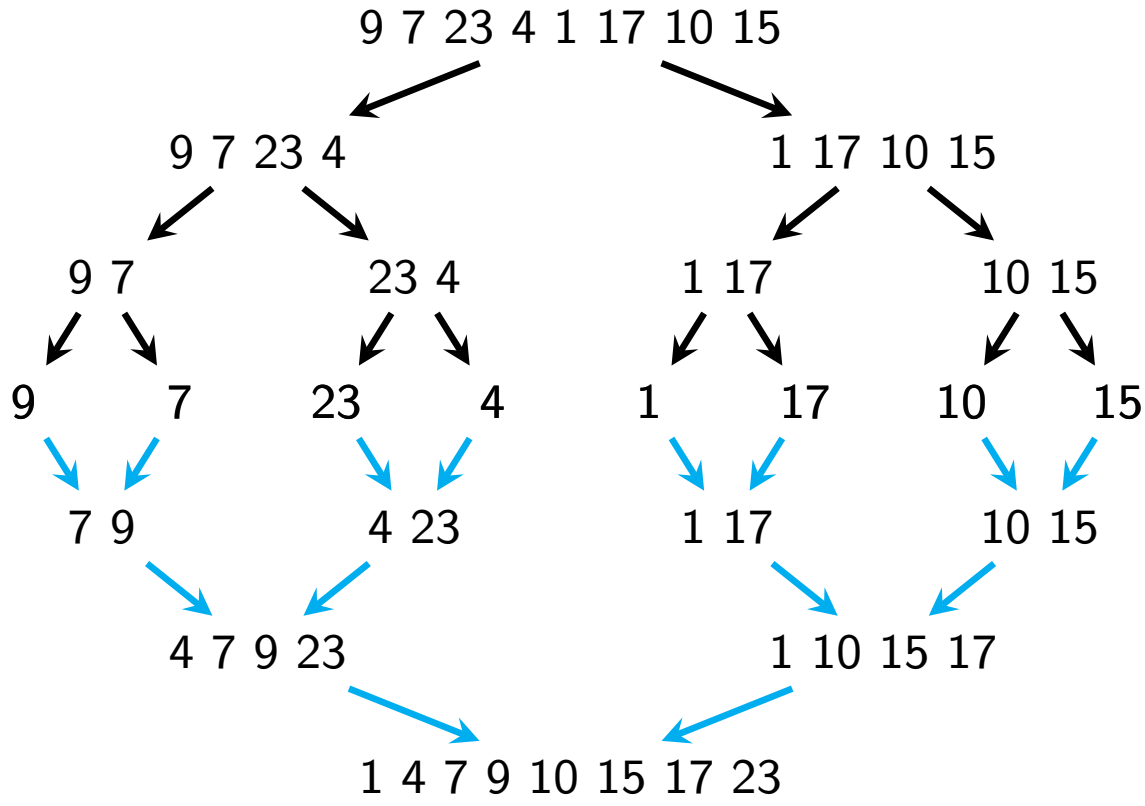
input : Dos listas ordenadas L_1 y L_2
output: Una lista ordenada L con los elementos de L_1 y L_2

```
1  Function merge (L1, L2)
2      L ← lista vacía
3      while L1 y L2 son ambas NO vacías do
4          remover el menor elemento m entre L1 y L2
5          agregar m al final de L
6      if L1 esta vacía then
7          Remover elementos de L2 y agregar al final de L
8      else if L2 esta vacía then
9          Remover elementos de L1 y agregar al final de L
10     return L
```

Merge es un algoritmo **de una pasada** por los datos

Merge sort: usado merge

¿cómo podemos ocupar merge para ordenar?



Merge sort: algoritmo

Algoritmo de Merge sort

input : Una lista de números $L = a_1 \dots a_n$

output: L en orden creciente

```
1 Function mergesort (L)
2     if n = 1 then
3         return L
4     else
5         m ← ⌊n/2⌋
6         L1 ← a1 a2 ... am
7         L2 ← am+1 am+2 ... an
8         return merge(mergesort(L1), mergesort(L2))
```

¿cuántas tiempo tardaremos en el **peor caso**?

$\mathcal{O}(n \cdot \log(n))$

Outline

Reglas de reescritura (anterior)

Operadores físicos

Merge sort

External merge sort

Optimizaciones

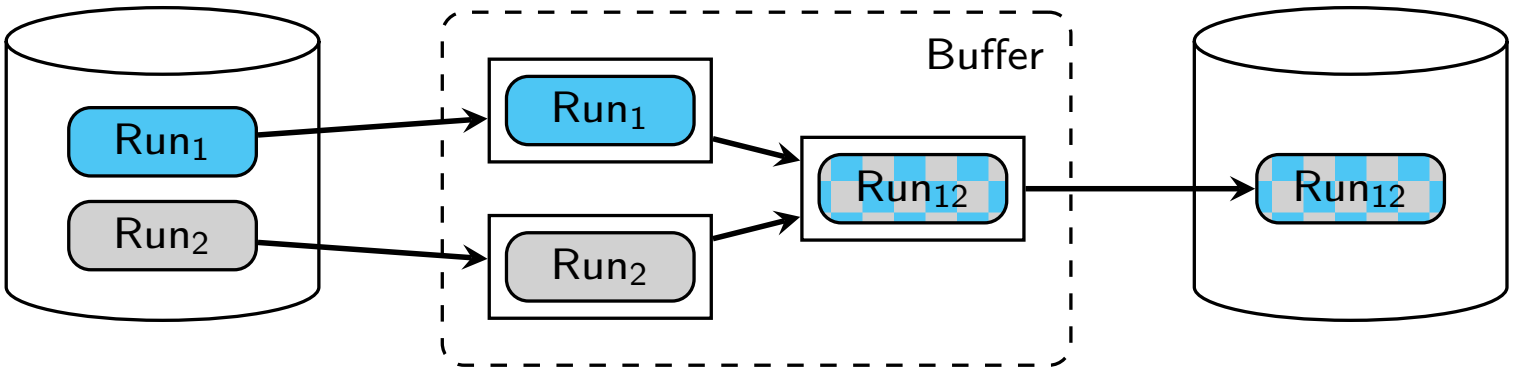
External Merge Sort

Definición

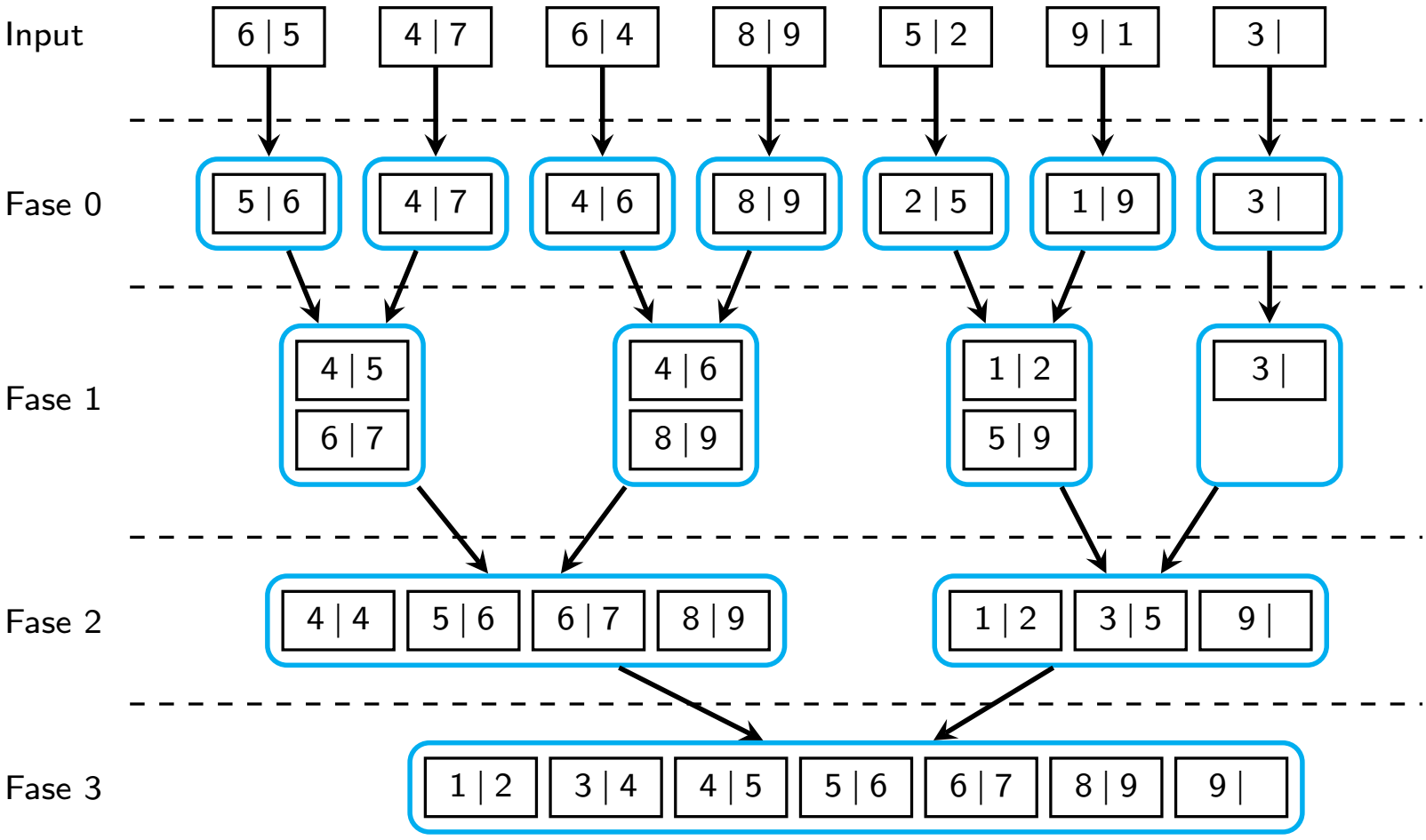
Run: secuencia de páginas que contiene una conjunto ordenado de tuplas.

External Merge Sort funciona en fases

- Fase 0: creamos los runs iniciales.
- Fase i:
  1. Traemos los runs desde el disco a memoria de a pares.
  2. Hacemos **merge** de cada par de runs.
  3. Almacenamos el nuevo run de regreso al disco.



External Merge Sort: ejemplo



External Merge Sort: algoritmo

Algoritmo de External Merge Sort

input : Páginas $P = \{p_1, \ldots, p_n\}$ con tuplas
output: Lista de páginas P' con las tuplas ordenadas

```
1  Function externalMergeSort(P)
2      foreach p_i ∈ P do
3          Leemos p_i a memoria
4          Ordenamos p_i en r_i
5          Escribimos r_i en disco
6      R ← {r_1, ..., r_n}
7      while |R| > 1 do
8          R' ← ∅
9          while R ≠ ∅ do
10             Leemos r'_1 y r'_2 a memoria        (R ← R \ {r'_1, r'_2})
11             Hacemos merge de r'_1 y r'_2        (r ← Merge(r'_1, r'_2))
12             Escribimos r en disco               (R' ← R' ∪ {r})
13         R ← R'
14     return único run en R
```

Eficiencia (en I/O) de External Merge Sort

Sea N el número de páginas del archivo.

- Cada fase lee y escribe cada página del archivo.
- ¿cuánto es el máximo **número de fases**?

$$\underbrace{1}_{\text{Fase } 0} + \underbrace{\lceil \log_2 N \rceil}_{\textit{Fases}}$$

- ¿cuánto es el **costo total** en I/O?

$$2 \cdot N \cdot (1 + \lceil \log_2 N \rceil)$$

Eficiencia (en I/O) de External Merge Sort

Costo en I/O

$$2 \cdot N \cdot (1 + \lceil \log_2 N \rceil)$$

Ejemplo

Considere un archivo de 8GB y páginas de 8 KB $\approx$ 1.048.576 páginas.

- Total de I/O $= 2 \cdot 1.048.576 \cdot (1 + \lceil \log_2 1.048.576 \rceil) = 44.040.192$ I/O.
- Si consideramos que cada I/O toma ≈ 0.1 ms (minimo):

  1.2 horas (tiempo total)

¿es posible **mejorar** este tiempo?

Outline

Reglas de reescritura (anterior)

Operadores físicos

Merge sort

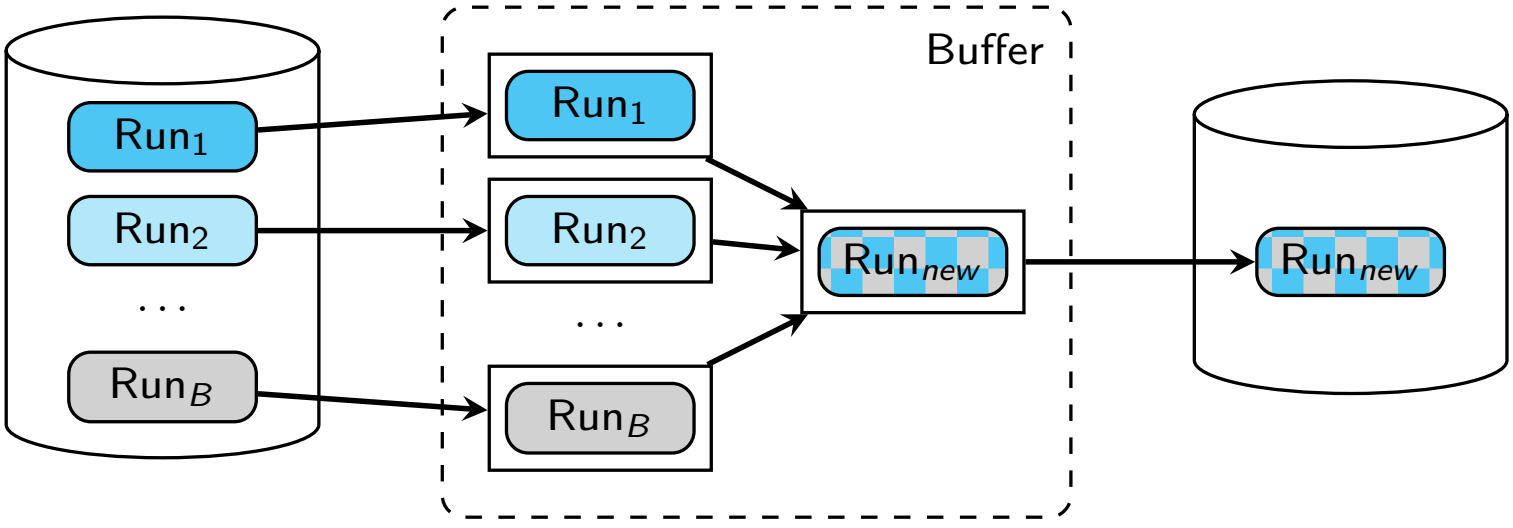
External merge sort

Optimizaciones

Optimizaciones para External Merge Sort

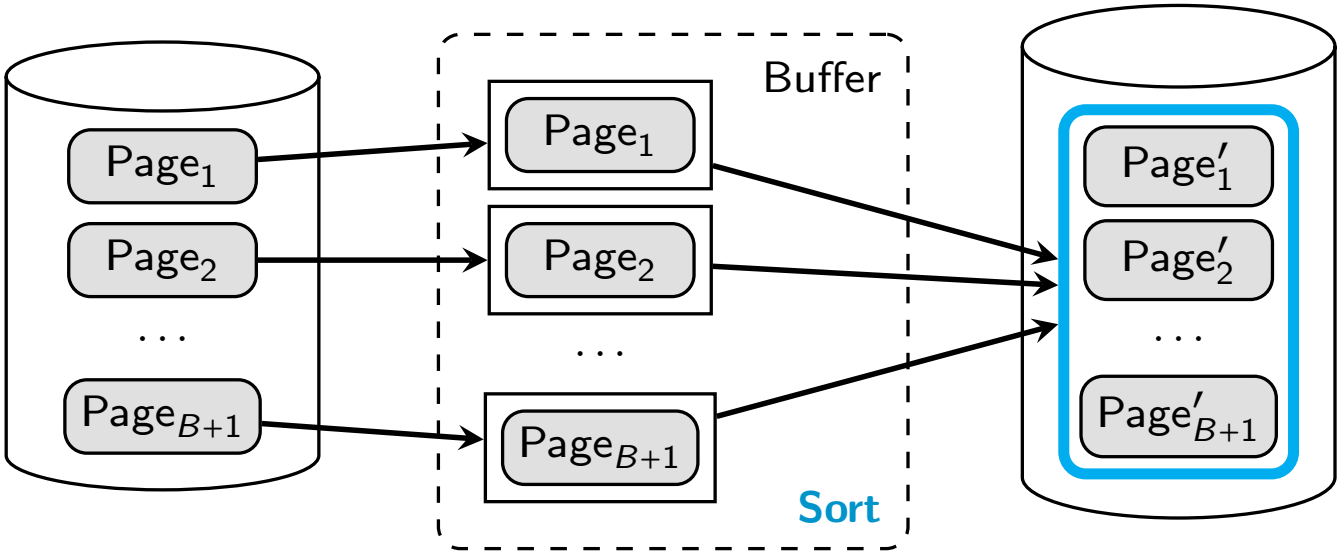
1. Aumentar el número de runs en el Merge.
2. Reducir el número de runs iniciales.
3. Evitar escribir el último resultado (**pipeline**).

1. Aumentar el número de runs en el Merge



Contamos con $B + 1$ páginas en buffer (en vez de 3)

2. Reducir el número de runs iniciales



Usamos las $B+1$ páginas en buffer
para crear **runs iniciales** con $B+1$ páginas

Combinando 1. y 2.

Algoritmo de External Merge Sort (optimizado)

input : Páginas $P = \{p_1, \ldots, p_n\}$ con tuplas y $B+1$ páginas en buffer
output: Lista de páginas P' con las tuplas ordenadas

1 **Function** `externalMergeSort` (P)
2 **while** $P \neq \emptyset$ **do**
3 Leemos $p'_1, \ldots, p'_{B+1}$ a memoria $(P \leftarrow P \setminus \{p'_1, \ldots, p'_{B+1}\})$
4 Ordenamos $p'_1, \ldots, p'_{B+1}$ en r_i
5 Escribimos r_i en disco
6 $R := \{r_1, \ldots, r_{\frac{n}{B+1}}\}$
7 **while** $|R| > 1$ **do**
8 $R' := \emptyset$
9 **while** $R \neq \emptyset$ **do**
10 Leemos $r'_1, \ldots, r'_B$ a memoria $(R \leftarrow R \setminus \{r'_1, \ldots, r'_B\})$
11 Hacemos merge de $r'_1, \ldots, r'_B$ $(r \leftarrow \text{Merge}(r'_1, \ldots, r'_B))$
12 Escribimos r en disco $(R' \leftarrow R' \cup \{r\})$
13 $R := R'$
14 **return** único run en R

Eficiencia (en I/O) de External Merge Sort Optimizado

Sea N el número de páginas del archivo y $B+1$ el tamaño del buffer.

- Cada fase lee y escribe cada página del archivo.
- ¿cuánto es el número de **runs iniciales**?

$$\left\lceil \frac{N}{B+1} \right\rceil$$

- ¿cuánto es el máximo **número de fases**?

$$\underbrace{1}_{\text{Fase } 0} + \underbrace{\left\lceil \log_B \left\lceil \frac{N}{B+1} \right\rceil \right\rceil}_{\textit{Fases}}$$

- ¿cuánto es el **costo total** en I/O?

$$2 \cdot N \cdot \left(1 + \left\lceil \log_B \left\lceil \frac{N}{B+1} \right\rceil \right\rceil\right)$$

Eficiencia (en I/O) de External Merge Sort Optimizado

Costo en I/O

$$2 \cdot N \cdot (1 + \left\lceil \log_B \left\lceil \frac{N}{B+1} \right\rceil \right\rceil)$$

Ejemplo

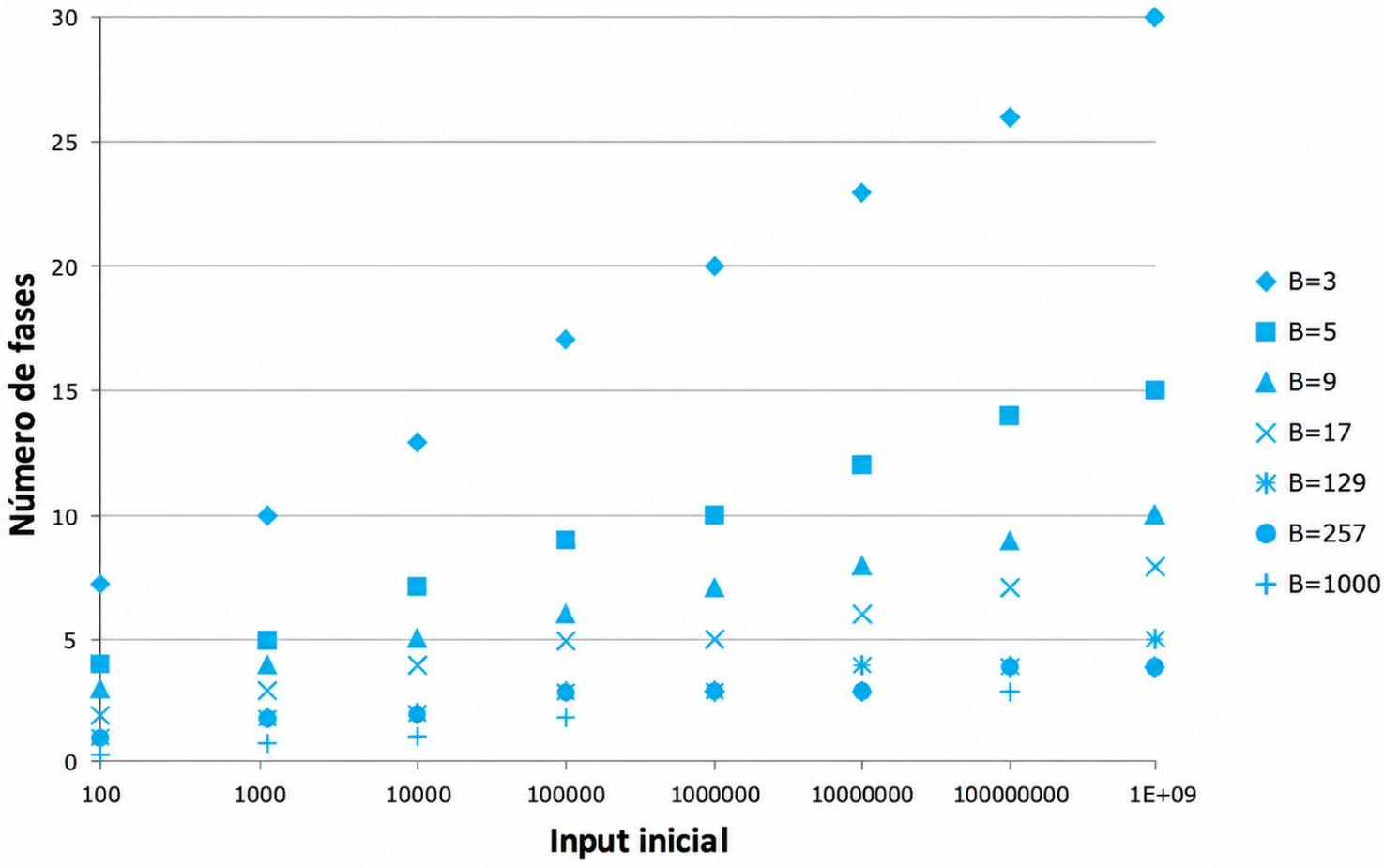
Considere:

1. el mismo archivo de 8GB y páginas de 8 KB $\approx$ 1.048.576 páginas.
2. **memoria ram** para el buffer de 2GB, $B + 1 \approx 262.145$ páginas.

Si consideramos que cada I/O toma ≈ 0.1 ms (minimo):

6.7 minutos

Número de fases según el tamaño del buffer



Número de fases según el tamaño del buffer

N	B=3	B=5	B=9	B=17	B=129	B=257	B=1000
100	7	4	3	2	1	1	1
10^3	10	5	4	3	2	2	1
10^4	13	7	5	4	2	2	2
10^5	17	9	6	5	3	3	2
10^6	20	10	7	5	3	3	3
10^7	23	12	8	6	4	3	3
10^8	26	14	9	7	4	4	3
10^9	30	15	10	8	5	4	4

- 10^9 páginas son 60 TB!
- 1000 páginas en buffer son solo 8 MB!

¿de qué tamaño debe ser el buffer para solo 2 fases?

$$2 = 1 + \left\lceil \log_B \left\lceil \frac{N}{B+1} \right\rceil \right\rceil$$



despejamos B

$$B \geq \sqrt{N}$$

10^9 páginas (60 TB): ¡solo necesitamos **240 MB de buffer**!

Conclusiones sobre el tamaño del buffer

En la práctica

Sea N = número de páginas del input:

1. Escogemos el tamaño del buffer (B) **dinámicamente** e igual a $\sqrt{N}$.
2. External Merge Sort solo tardará **dos fases**.

Costo de External Merge Sort $\approx 4 \cdot N$

Optimizaciones para External Merge Sort

1. Aumentar el número de runs en el Merge. ✓
2. Reducir el número de runs iniciales. ✓
3. Evitar escribir el último resultado (**pipeline**).

3. Evitar escribir el último resultado

Si consideramos que:

1. **Sorting** es una operación mas en nuestro plan físico.
2. Tuplas ordenadas son recibidas por otro operador (**pipeline**).

¡podemos evitar escribir el último resultado en disco!

Costo de External Merge Sort $\approx 3 \cdot N$